

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.25

EMBEDDED POSTGRESQL

 Copy as Markdown

Embedded PostgreSQL

New projects run an embedded PostgreSQL in the `development` and `test` run modes. It is a real PostgreSQL server that Serverpod starts and stops together with your server, so local development gets the same database engine as production without installing PostgreSQL or running Docker.

It is meant for local development and testing only. There are no backups, no replication, and nothing supervising the process, so staging and production connect to a managed or separately operated PostgreSQL instead. See [Connection](#) for those setups.

Enable it

Embedded PostgreSQL is enabled by the `dataPath` setting, which generated projects already include:

config/development.yaml

```
database:
  host: localhost
  port: 8090
  name: myproject
  user: postgres
  dataPath: .serverpod/development/pgdata
```

The rest of the database settings stay as they are. Serverpod uses `name` and `user` when it creates the database and reads the password from `config/passwords.yaml`, exactly as it would for an external instance. Remove `dataPath` to connect to the `host` and `port` instead.

A relative `dataPath` is resolved from the root of the server package, and each run mode needs its own directory, for example `.serverpod/test/pgdata` in `config/test.yaml`. The directory holds a complete database, so keep it out of version control.



What happens when the server starts

The first start downloads the PostgreSQL binaries for your operating system and architecture into a per-user cache, which every later start and every other project reuses. Binaries are available for Linux and macOS on x64 and Arm64, and for Windows on x64.

The cache location depends on your operating system:

- **macOS:** `~/Library/Caches/serverpod/pg-binaries`
- **Linux:** `$XDG_CACHE_HOME/serverpod/pg-binaries`, or `~/.cache/serverpod/pg-binaries` when `XDG_CACHE_HOME` is not set
- **Windows:** `%LOCALAPPDATA%\serverpod\Cache\pg-binaries`

Set `SERVERPOD_PG_CACHE_DIR` to keep the binaries in a different directory.

Serverpod starts the database before it opens its connection pool, and connects over a Unix domain socket instead of a TCP port, so several projects can run at once without competing for ports. If another Serverpod process already runs a database in the same `dataPath`, the new process attaches to it rather than starting a second one.

The files under `dataPath` are kept when the server stops, so your data is still there on the next start. After an unclean exit, Serverpod clears the leftover process state and brings the database back up.

Connect a database tool

Most clients like `psql` and `pgAdmin` connect over TCP, not the Unix socket the server uses. To inspect the data with one of them, start the database on its own while the server is stopped:

```
$ serverpod database start
```

It boots the database configured for the `development` run mode and keeps it listening on the configured port until you stop it with Ctrl+C. Connect with the `name` and `user` from that run mode's configuration, and the password from `config/passwords.yaml`.

Pass `--mode` to pick another run mode and `--port` to listen somewhere else:

```
$ serverpod database start --mode test --port 9090
```

Run integration tests

Integration tests bring their own database, so there is nothing to install or start first:

```
$ dart test
```

The started database server will be held open until the test process exits. Every suite in a test process shares the same database server, but each `withServerpod` group creates a randomly named database of its own on it that will be dropped on teardown. That is where the isolation comes from: groups run in parallel without seeing each other's data or leaving anything behind between runs.

The data directory is not temporary. Like the one for `development`, it stays after the tests, and only the server process is reclaimed when the test process exits. See [Get started with testing](#) for the rest of the setup.

NOTE

If a test process is hard-killed, the database server will not be stopped and data can be left behind. This data won't harm future runs because of the named database isolation, but it can occupy disk space. If the directory happens to grow too large, it is fully safe to delete it and let the next test recreate it from scratch.

Reset the database

To start from an empty database, stop the server and delete the directory `dataPath` points at. Serverpod creates a new one on the next start.

WARNING

Deleting `dataPath` permanently deletes the database and all local data stored in it.

Related

- [Connection](#): connecting to a PostgreSQL instance you run yourself.
- [Configuration](#): the database section of the run mode configuration.
- [serverpod database](#): every option of the command above.
- [serverpod_embedded_postgres](#): the package behind this, for tools that need a PostgreSQL process of their own.